regularization_synthetic

April 1, 2026

1 Regularization: Penalized regression

Regularization force the learning algorithm to build a less complex model . In practice, that often leads to slightly higher bias but significantly reduces the variance. The most common penalized regression models are:

1. Ridge regression
2. Lasso regression
3. Elastic Net regression

$$y = f(x) = x + 2x^2 - 3x^3 + \epsilon$$

The target variable is **y** and the features are 5 polynomial features of **X**. Namely, $X_1 = X$, $X_2 = X^2$, $X_3 = X^3$ and etc.

First we need to import the libraries:

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
import warnings
warnings.filterwarnings('ignore')
plt.style.use("fivethirtyeight")
plt.rcParams["figure.figsize"] = (14,6)
plt.rcParams.update({'font.size': 12, 'axes.labelsize': 14, 'axes.titlesize':
 ↪16})
plt.rcParams['figure.dpi'] = 120
```

[2]:
```
rand_state= 1000
```

[3]:
```
#  load the dataset
df = pd.read_csv(r"D:\datasets\Regularization_synthetic.csv")
# load the data
df.head()
```

[3]:

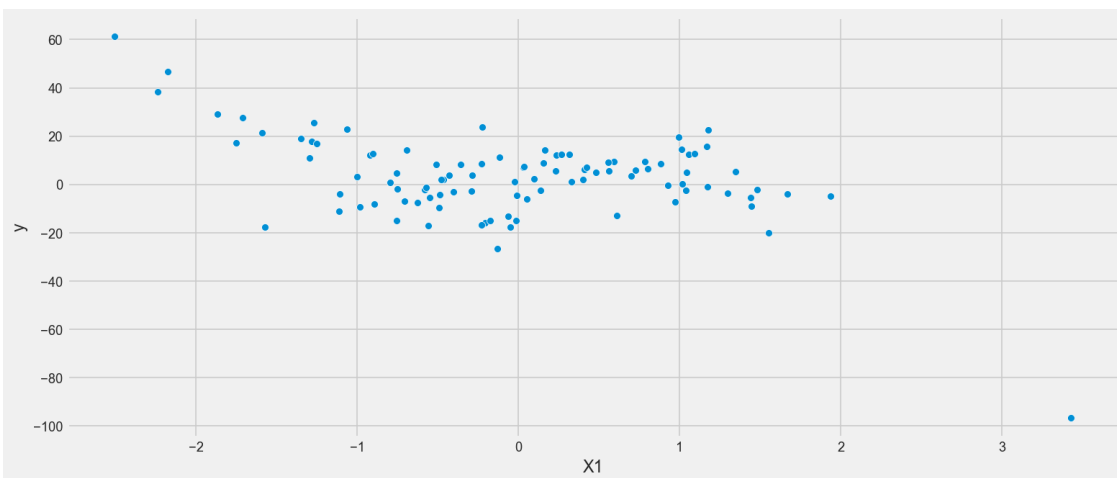
	X1	X2	X3	X4	X5	y
0	1.668068	2.782452	4.641320	7.742038	12.914249	-3.901562
1	0.925862	0.857220	0.793667	0.734826	0.680348	-0.384578
2	1.057997	1.119357	1.184276	1.252960	1.325628	12.493677
3	-0.920339	0.847024	-0.779549	0.717449	-0.660297	12.237389
4	1.299748	1.689346	2.195725	2.853890	3.709339	-3.772851

[4]:
```
df.head()
```

[4]:

	X1	X2	X3	X4	X5	y
0	1.668068	2.782452	4.641320	7.742038	12.914249	-3.901562
1	0.925862	0.857220	0.793667	0.734826	0.680348	-0.384578
2	1.057997	1.119357	1.184276	1.252960	1.325628	12.493677
3	-0.920339	0.847024	-0.779549	0.717449	-0.660297	12.237389
4	1.299748	1.689346	2.195725	2.853890	3.709339	-3.772851

[5]:
```
sns.scatterplot(x='X1', y='y', data=df)
plt.show()
```



important: we need to normalize the features before training the models.

[6]:
```
from sklearn.preprocessing import StandardScaler
```

[7]:
```
scaler=StandardScaler()
df_sc= scaler.fit_transform(df)
```

[8]:
```
df_sc.shape
```

[8]: (100, 6)

[9]:
```
df_sc[:2]
```

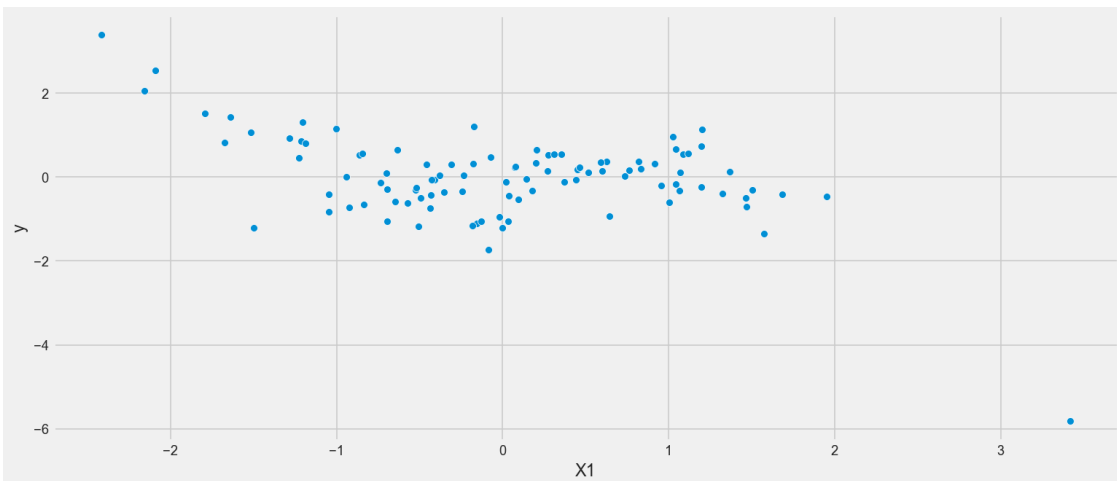
[9]:
```
array([[ 1.68559489,  1.10379599,  0.93969261,  0.28529469,  0.20893775,
        -0.41160092],
       [ 0.95551657, -0.1126954 ,  0.15582326, -0.19468946, -0.0400858 ,
        -0.20670883]])
```

[10]:
```python
df_sc = pd.DataFrame(df_sc, columns=df.columns)
df_sc.head()
```

[10]:

	X1	X2	X3	X4	X5	y
0	1.685595	1.103796	0.939693	0.285295	0.208938	-0.411601
1	0.955517	-0.112695	0.155823	-0.194689	-0.040086	-0.206709
2	1.085492	0.052941	0.235401	-0.159198	-0.026951	0.543551
3	-0.860516	-0.119138	-0.164683	-0.195880	-0.067375	0.528621
4	1.323294	0.413098	0.441460	-0.049537	0.021570	-0.404102

[11]:
```python
sns.scatterplot(x='X1', y='y', data=df_sc)
plt.show()
```



1.1 Splitting the data

[12]:
```python
y = df_sc['y']
X = df_sc.drop('y', axis=1)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=rand_state)
X_train.head()
```

[12]:

	X1	X2	X3	X4	X5
37	1.464631	0.662152	0.606819	0.052327	0.073610
44	1.572794	0.870372	0.757779	0.153824	0.130178

```
55  0.275319 -0.619639 -0.003245 -0.244817 -0.053920
66 -0.493702 -0.464986 -0.039290 -0.238872 -0.054935
82  0.272588 -0.620456 -0.003337 -0.244827 -0.053921
```

Let's first run a linear regression (using statsmodels.api) and see the results as a benchmark.

[13]:
```
import statsmodels.api as sm
```

[14]:
```
X_test_wc = sm.add_constant(X_test)
X_train_wc = sm.add_constant(X_train)
```

[15]:
```
model = sm.OLS(y_train, X_train_wc).fit()
model.summary()
```

[15]:

Dep. Variable:	y	**R-squared:**	0.451
Model:	OLS	**Adj. R-squared:**	0.414
Method:	Least Squares	**F-statistic:**	12.17
Date:	Wed, 01 Apr 2026	**Prob (F-statistic):**	1.28e-08
Time:	06:45:04	**Log-Likelihood:**	-65.981
No. Observations:	80	**AIC:**	144.0
Df Residuals:	74	**BIC:**	158.3
Df Model:	5		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	0.0870	0.173	0.504	0.616	-0.257	0.431
X1	0.5370	0.199	2.705	0.008	0.141	0.933
X2	0.1037	0.302	0.343	0.732	-0.498	0.705
X3	-2.1339	0.839	-2.543	0.013	-3.806	-0.462
X4	0.2535	0.861	0.294	0.769	-1.463	1.970
X5	1.5933	1.669	0.954	0.343	-1.733	4.920

Omnibus:	11.757	**Durbin-Watson:**	2.213
Prob(Omnibus):	0.003	**Jarque-Bera (JB):**	14.171
Skew:	-0.701	**Prob(JB):**	0.000837
Kurtosis:	4.512	**Cond. No.**	33.9

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In this section we will train 4 different models: 1. Linear regression (model_linear) 2. Ridge regression (model_ridge) 3. Lasso regression (model_lasso) 4. Elastic Net regression (model_net)

[16]:
```
from sklearn.linear_model import LinearRegression, Ridge,RidgeCV, Lasso,␣
↪LassoCV, ElasticNet, ElasticNetCV
```

[17]:
```
model_linear = LinearRegression()
model_ridge = Ridge(alpha=0)
```

```
model_lasso = Lasso(alpha=0)
model_net = ElasticNet(alpha=0)
```

[18]:
```
y_hat_linear= model_linear.fit(X_train, y_train).predict(X_test)
y_hat_ridge = model_ridge.fit(X_train, y_train).predict(X_test)
y_hat_lasso = model_lasso.fit(X_train, y_train).predict(X_test)
y_hat_net   = model_net.fit(X_train, y_train).predict(X_test)
```

[19]:
```
df_predictions = pd.DataFrame({'y_test':y_test,
                               'y_hat_linear':y_hat_linear,
                               'y_hat_ridge':y_hat_ridge,
                               'y_hat_lasso':y_hat_lasso,
                               'y_hat_net':y_hat_net})
df_predictions.head()
```

[19]:

	y_test	y_hat_linear	y_hat_ridge	y_hat_lasso	y_hat_net
88	1.307243	0.171067	0.171067	0.171067	0.171067
4	-0.404102	-0.079686	-0.079686	-0.079686	-0.079686
62	-1.055680	-0.180921	-0.180921	-0.180921	-0.180921
63	2.538916	2.344482	2.344482	2.344482	2.344482
48	1.144605	-0.078630	-0.078630	-0.078630	-0.078630

1.1.1 coefficients

[20]:
```
df.drop('y', axis=1, inplace=False).columns
```

[20]:
```
Index(['X1', 'X2', 'X3', 'X4', 'X5'], dtype='object')
```

[21]:
```
coefficients = pd.DataFrame({'Features':df.drop('y', axis=1, inplace=False).
 ↪columns})
coefficients['model_lin']= model_linear.coef_
coefficients['model_ridge']= model_ridge.coef_
coefficients['model_lasso']= model_lasso.coef_
coefficients['model_net']= model_net.coef_
coefficients
```

[21]:

	Features	model_lin	model_ridge	model_lasso	model_net
0	X1	0.537047	0.537047	0.537047	0.537047
1	X2	0.103655	0.103655	0.103655	0.103655
2	X3	-2.133872	-2.133872	-2.133872	-2.133872
3	X4	0.253492	0.253492	0.253492	0.253492
4	X5	1.593336	1.593336	1.593336	1.593336

1.1.2 Performance in the test set

[22]:
```
model_linear.score(X_test, y_test)
```

[22]: -0.5051758896585028

[23]:
```
MSE_test_lin = np.mean(np.square(df_predictions['y_test'] -␣
↪df_predictions['y_hat_linear']))
RMSE_test_lin = np.sqrt(MSE_test_lin)
np.round(RMSE_test_lin,3)
```

[23]: np.float64(2.037)

[24]:
```
model_ridge.score(X_test, y_test)
```

[24]: -0.5051758896584939

[25]:
```
MSE_test_ridge = np.mean(np.square(df_predictions['y_test'] -␣
↪df_predictions['y_hat_ridge']))
RMSE_test_ridge = np.sqrt(MSE_test_ridge)
np.round(RMSE_test_ridge,3)
```

[25]: np.float64(2.037)

[26]:
```
model_lasso.score(X_test, y_test)
```

[26]: -0.5051758639724284

[27]:
```
MSE_test_lasso = np.mean(np.square(df_predictions['y_test'] -␣
↪df_predictions['y_hat_lasso']))
RMSE_test_lasso = np.sqrt(MSE_test_lasso)
np.round(RMSE_test_lasso,3)
```

[27]: np.float64(2.037)

[28]:
```
model_net.score(X_test, y_test)
```

[28]: -0.5051758639724284

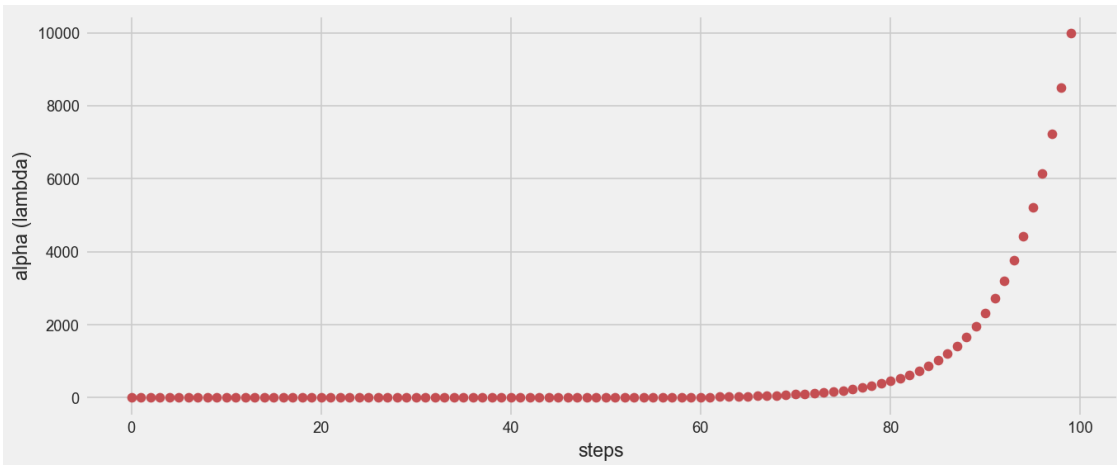
[29]:
```
MSE_test_net = np.mean(np.square(df_predictions['y_test'] -␣
↪df_predictions['y_hat_net']))
RMSE_test_net = np.sqrt(MSE_test_net)
np.round(RMSE_test_net,3)
```

[29]: np.float64(2.037)

1.1.3 Plotting the regression coefficients vs alphas:

1) Ridge regression coefficients vs alpha

[30]:
```
alpha_ridge = 10**np.linspace(-3,4,100)
plt.figure(figsize=(12,5))
plt.plot(alpha_ridge,'or' )
plt.xlabel('steps')
plt.ylabel('alpha (lambda)')
plt.show()
```



[31]:
```
len(alpha_ridge)
```

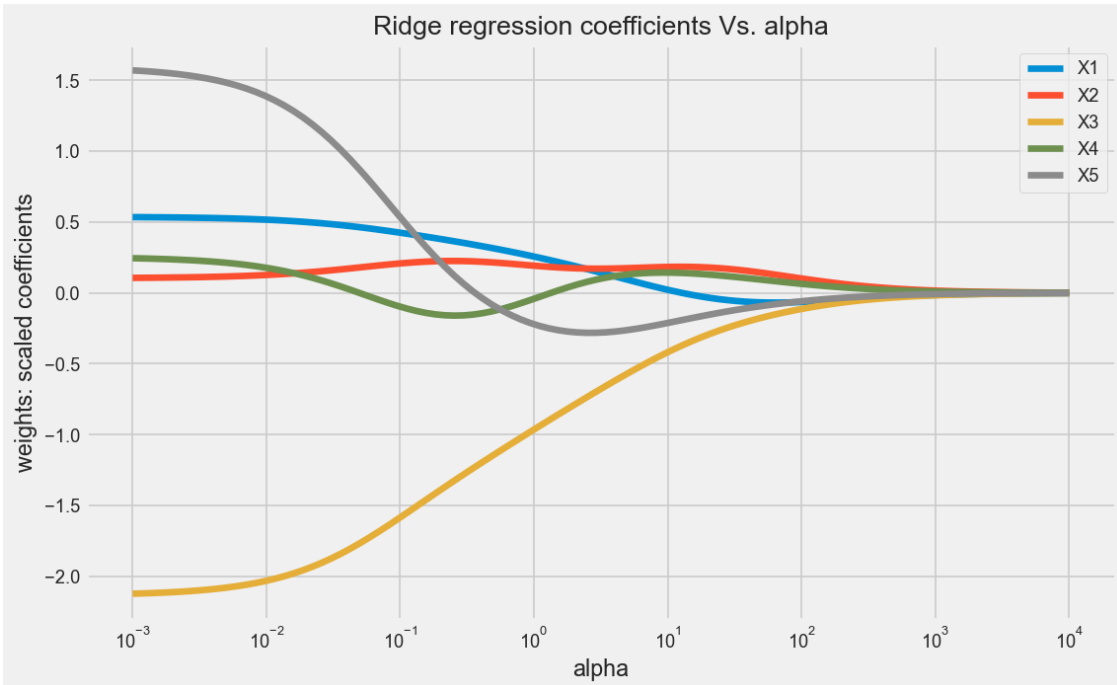
[31]: 100

[32]:
```
ridge = Ridge()
coefs_ridge = []

for i in alpha_ridge:
    ridge.set_params(alpha = i)
    ridge.fit(X_train, y_train)
    coefs_ridge.append(ridge.coef_)

np.shape(coefs_ridge)
```

[32]: (100, 5)

[33]:
```
plt.figure(figsize=(10,6))
ax = plt.gca()
ax.plot(alpha_ridge, coefs_ridge)
ax.set_xscale('log')
plt.axis('tight')
plt.xlabel('alpha')
```

```
plt.ylabel('weights: scaled coefficients')
plt.title('Ridge regression coefficients Vs. alpha')
plt.legend(df.drop('y',axis=1, inplace=False).columns)

plt.show()
```



---

2) Lasso regression coefficients vs alpha

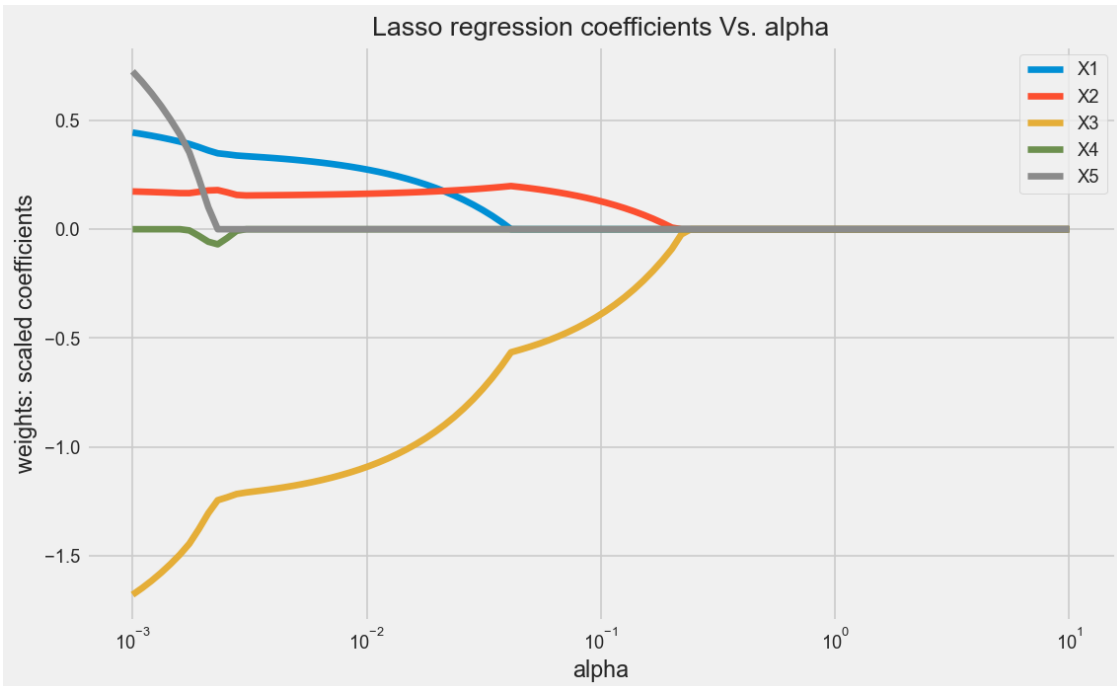
[34]:
```
alpha_lasso = 10**np.linspace(-3,1,100)
```

[35]:
```
lasso = Lasso()
coefs_lasso = []

for i in alpha_lasso:
    lasso.set_params(alpha = i)
    lasso.fit(X_train, y_train)
    coefs_lasso.append(lasso.coef_)

np.shape(coefs_lasso)
```

[35]: (100, 5)

[36]:
```
plt.figure(figsize=(10,6))
ax = plt.gca()
```

```
ax.plot(alpha_lasso, coefs_lasso)
ax.set_xscale('log')
plt.axis('tight')
plt.xlabel('alpha')
plt.ylabel('weights: scaled coefficients')
plt.title('Lasso regression coefficients Vs. alpha')
plt.legend(df.drop('y',axis=1, inplace=False).columns)

plt.show()
```



---

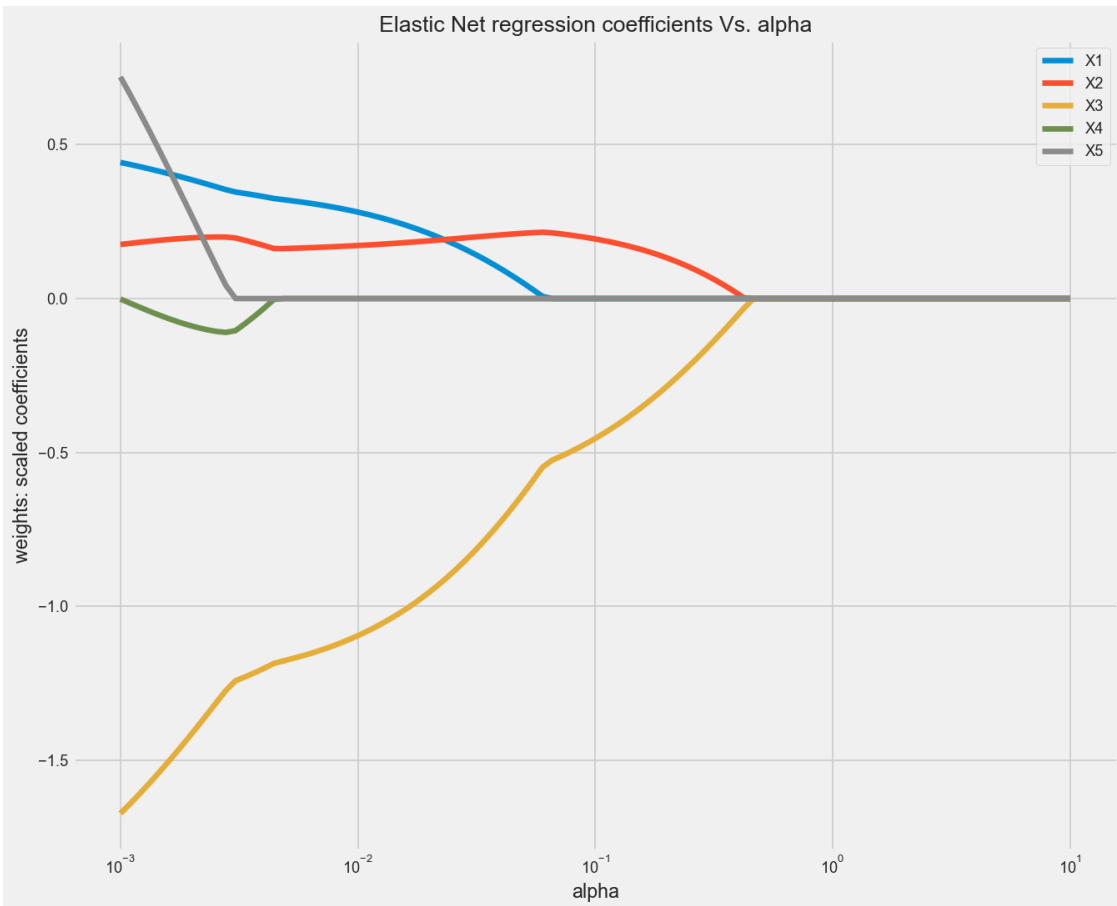
3) Elastic Net regression coefficients vs alpha

[37]:
```
alpha_elasticnet = 10**np.linspace(-3,1,100)
```

[38]:
```
elasticnet = ElasticNet()
coefs_elasticnet = []

for i in alpha_elasticnet:
    elasticnet.set_params(alpha = i)
    elasticnet.fit(X_train, y_train)
    coefs_elasticnet.append(elasticnet.coef_)

np.shape(coefs_elasticnet)
```

[38]: (100, 5)

[39]:
```
plt.figure(figsize=(12,10))
ax = plt.gca()
ax.plot(alpha_elasticnet, coefs_elasticnet)
ax.set_xscale('log')
plt.axis('tight')
plt.xlabel('alpha')
plt.ylabel('weights: scaled coefficients')
plt.title('Elastic Net regression coefficients Vs. alpha')
plt.legend(df.drop('y',axis=1, inplace=False).columns)

plt.show()
```



1.2 Cross Validation

Let's use cross validation to find the optimal Lambdas (alphas) for different models.

1) Optimal alpha for Ridge regression

[40]:
```
ridgecv = RidgeCV(alphas=alpha_ridge)
ridgecv.fit(X_train, y_train)
ridgecv.alpha_
```

[40]:
```
np.float64(1.291549665014884)
```

[41]:
```
alpha_ridge_opt = ridgecv.alpha_
```

[42]:
```
ridgecv.coef_
```

[42]:
```
array([ 0.23326653,  0.18398615, -0.89894889, -0.00492102, -0.24987795])
```

---

2) Optimal alpha for Lasso regression

[43]:
```
lassocv = LassoCV(alphas=alpha_lasso)
lassocv.fit(X_train, y_train)
lassocv.alpha_
```

[43]:
```
np.float64(0.004862601580065354)
```

[44]:
```
alpha_lasso_opt = lassocv.alpha_
```

[45]:
```
lassocv.coef_
```

[45]:
```
array([ 0.31944719,  0.15721496, -1.17879735, -0.        ,  0.        ])
```

---

3) Optimal alpha for Elastic Net regression

[46]:
```
elasticnetcv = ElasticNetCV(alphas=alpha_elasticnet, l1_ratio=np.linspace(0.0,␣
↪1, 11))
elasticnetcv.fit(X_train, y_train)
elasticnetcv.alpha_
```

[46]:
```
np.float64(0.004862601580065354)
```

[47]:
```
elasticnetcv.l1_ratio_
```

[47]:
```
np.float64(0.9)
```

[48]:
```
alpha_elasticnet_opt = elasticnetcv.alpha_
```

[49]:
```
elasticnetcv.coef_
```

[49]:
```
array([ 0.31976495,  0.15827089, -1.17839143, -0.        ,  0.        ])
```